🧠 BÀI THỰC HÀNH MEMENTO PATTERN

✅ Backend: ASP.NET Core Web API
✅ Frontend: React (gọi API Undo/Redo)
✅ Áp dụng **Memento Pattern đúng chuẩn GoF**
✅ Có xử lý **Undo / Redo trạng thái Order**

🎯 BÀI TOÁN

Hệ thống E-commerce cho phép người dùng:

- Tạo đơn hàng
- Thay đổi trạng thái đơn hàng (Pending → Paid → Shipped)
- **Undo / Redo trạng thái đơn hàng**

👉 Ví dụ:

- User chuyển: Pending → Paid → Shipped
- Sau đó bấm Undo → quay lại Paid
- Undo tiếp → quay lại Pending

📦 KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
        ↓
POST /api/orders/update-status
POST /api/orders/undo
POST /api/orders/redo
        ↓
OrderController
        ↓
OrderService
        ↓
Order (Originator)
        ↓
OrderMemento
        ↓
OrderCaretaker (Undo/Redo stack)
```

💻 BACKEND – ASP.NET Core 8 Web API

1️⃣ Tạo project

```
dotnet new webapi -n MementoOrderDemo
cd MementoOrderDemo
```

🗂 Cấu trúc thư mục

```
MementoOrderDemo
 ├── Controllers
 │     └── OrdersController.cs
 ├── Services
 │     └── OrderService.cs
 ├── Memento
 │     ├── OrderMemento.cs
 │     ├── OrderCaretaker.cs
 ├── Models
 │     ├── Order.cs
 │     ├── UpdateStatusRequest.cs
 │     └── OrderResponse.cs
 └── Program.cs
```

💮 1. ORIGINATOR (ORDER)

📌 Models/Order.cs

```
using MementoOrderDemo.Memento;

namespace MementoOrderDemo.Models;

public class Order
{
    public string Status { get; private set; } =
"Pending";

    public void SetStatus(string status)
    {
        Status = status;
    }

    // Save state
    public OrderMemento Save()
    {
        return new OrderMemento(Status);
```

```
    }

    // Restore state
    public void Restore(OrderMemento memento)
    {
        Status = memento.Status;
    }
}
```

🏵️ 2. MEMENTO

📌 Memento/OrderMemento.cs

```
namespace MementoOrderDemo.Memento;

public class OrderMemento
{
    public string Status { get; }

    public OrderMemento(string status)
    {
        Status = status;
    }
}
```

🏵️ 3. CARETAKER (QUẢN LÝ UNDO/REDO)

📌 Memento/OrderCaretaker.cs

```
using MementoOrderDemo.Models;

namespace MementoOrderDemo.Memento;

public class OrderCaretaker
{
    private readonly Stack<OrderMemento> _undoStack =
new();
    private readonly Stack<OrderMemento> _redoStack =
new();

    public void SaveState(Order order)
    {
        _undoStack.Push(order.Save());
        _redoStack.Clear();
    }
```

```
    public void Undo(Order order)
    {
        if (_undoStack.Count == 0) return;

        var current = order.Save();
        _redoStack.Push(current);

        var prev = _undoStack.Pop();
        order.Restore(prev);
    }

    public void Redo(Order order)
    {
        if (_redoStack.Count == 0) return;

        var current = order.Save();
        _undoStack.Push(current);

        var next = _redoStack.Pop();
        order.Restore(next);
    }
}
```

---

4. SERVICE

📌 Services/OrderService.cs

```
using MementoOrderDemo.Models;
using MementoOrderDemo.Memento;

namespace MementoOrderDemo.Services;

public class OrderService
{
    private readonly Order _order;
    private readonly OrderCaretaker _caretaker;

    public OrderService()
    {
        _order = new Order();
        _caretaker = new OrderCaretaker();
    }

    public string GetStatus()
    {
```

```
        return _order.Status;
    }

    public string UpdateStatus(string newStatus)
    {
        _caretaker.SaveState(_order);

        _order.SetStatus(newStatus);

        return _order.Status;
    }

    public string Undo()
    {
        _caretaker.Undo(_order);
        return _order.Status;
    }

    public string Redo()
    {
        _caretaker.Redo(_order);
        return _order.Status;
    }
}
```

🏵️ 5. REQUEST MODEL

📌 Models/UpdateStatusRequest.cs

```
namespace MementoOrderDemo.Models;

public class UpdateStatusRequest
{
    public string Status { get; set; }
}
```

📌 Models/OrderResponse.cs

```
namespace MementoOrderDemo.Models;

public class OrderResponse
{
    public string Status { get; set; }
}
```

---

6. CONTROLLER

Controllers/OrdersController.cs

```
using Microsoft.AspNetCore.Mvc;
using MementoOrderDemo.Models;
using MementoOrderDemo.Services;

namespace MementoOrderDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _service;

    public OrdersController(OrderService service)
    {
        _service = service;
    }

    [HttpPost("update-status")]
    public IActionResult Update(UpdateStatusRequest
request)
    {
        var status =
_service.UpdateStatus(request.Status);
        return Ok(new OrderResponse { Status = status });
    }

    [HttpPost("undo")]
    public IActionResult Undo()
    {
        var status = _service.Undo();
        return Ok(new OrderResponse { Status = status });
    }

    [HttpPost("redo")]
    public IActionResult Redo()
    {
        var status = _service.Redo();
        return Ok(new OrderResponse { Status = status });
    }

    [HttpGet]
    public IActionResult Get()
    {
```

```
        return Ok(new OrderResponse { Status =
_service.GetStatus() });
    }
}
```

---

7. ĐĂNG KÝ DI

📌 Program.cs

```
using MementoOrderDemo.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddSingleton<OrderService>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                  .AllowAnyHeader()
                  .AllowAnyMethod();
        });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseCors("AllowReact");
app.MapControllers();

app.Run();
```

▶ CHẠY BACKEND

```
dotnet run
```

Test API:

Update status

```
POST /api/orders/update-status
{
  "status": "Paid"
}
```

Undo

```
POST /api/orders/undo
```

Redo

```
POST /api/orders/redo
```

💻 FRONTEND – REACT

Tạo project

```
npx create-react-app memento-ui
cd memento-ui
npm install axios
```

---

src/App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [status, setStatus] = useState("");
  const [result, setResult] = useState("");

  const update = async () => {
    const res = await
axios.post("https://localhost:xxxx/api/orders/update-
status", {
      status,
    });
    setResult(res.data.status);
  };
```

```
  const undo = async () => {
    const res = await
axios.post("https://localhost:xxxx/api/orders/undo");
    setResult(res.data.status);
  };

  const redo = async () => {
    const res = await
axios.post("https://localhost:xxxx/api/orders/redo");
    setResult(res.data.status);
  };

  return (
    <div style={{ padding: 40 }}>
      <h2>Order Status</h2>

      <input
        placeholder="Enter status (Pending/Paid/Shipped)"
        onChange={(e) => setStatus(e.target.value)}
      />

      <br /><br />

      <button onClick={update}>Update</button>
      <button onClick={undo}>Undo</button>
      <button onClick={redo}>Redo</button>

      <h3>Current: {result}</h3>
    </div>
  );
}

export default App;
```

▶ CHẠY FRONTEND

```
npm start
```

🔥 LUỒNG XỬ LÝ MEMENTO

👉 Khi Update Status

1. User nhập: `Paid`
2. Gọi API `/update-status`
3. Service:

- Save trạng thái hiện tại → `_caretaker.SaveState()`
- Update trạng thái mới

👉 Khi Undo

1. Gọi `/undo`
2. Caretaker:
   - Lấy state cũ từ `_undoStack`
   - Push state hiện tại vào `_redoStack`
   - Restore

👉 Khi Redo

1. Gọi `/redo`
2. Caretaker:
   - Lấy state từ `_redoStack`
   - Restore lại

🎯 TỔNG KẾT (GOF CHUẨN)

Thành phần	Vai trò
Originator	Order
Memento	OrderMemento
Caretaker	OrderCaretaker
Client	OrderService

🚀 MỞ RỘNG BÀI TẬP

Cho sinh viên:

1. Lưu nhiều field (Status + TotalPrice)
2. Persist history vào DB
3. Giới hạn số bước Undo (max 10)
4. Áp dụng cho:
   - Text Editor
   - Shopping Cart
   - Form builder